ProxySQL Complete Documentation

Table of Contents

1. Introduction
2. Installation
3. Configuration
4. Admin Interface
5. Query Rules
6. Connection Pooling
7. Load Balancing
8. Monitoring
9. Advanced Features
10. Troubleshooting

Introduction

What is ProxySQL?

ProxySQL is a high-performance SQL proxy that sits between applications and MySQL/MariaDB servers. It provides:

- **Connection Pooling**: Reuses database connections to reduce overhead
- **Query Routing**: Routes queries to appropriate backends based on rules
- **Load Balancing**: Distributes connections across multiple servers
- **Query Caching**: Caches query results for improved performance
- **Query Filtering**: Validates and blocks dangerous queries
- **Traffic Mirroring**: Duplicates queries to test environments

Key Features

- Lightweight and fast (written in C/C++)
- No changes needed to application code

- Transparent to the application layer
- Advanced query routing with regex patterns
- Built-in admin interface for configuration
- Statistics and monitoring capabilities
- Support for multiple backend servers
- Session multiplexing

Installation

macOS

```
# Using Homebrew
brew install proxysql

# Verify installation
proxysql --version
```

Linux (Ubuntu/Debian)

```
# Download and install
wget https://github.com/sysown/proxysql/releases/download/v2.5.4/proxysql_2.5.4_ubuntu2
sudo dpkg -i proxysql_2.5.4_ubuntu22_amd64.deb

# Start service
sudo systemctl start proxysql
sudo systemctl enable proxysql

# Check status
sudo systemctl status proxysql
```

Linux (CentOS/RHEL)

```
# Download and install
wget https://github.com/sysown/proxysql/releases/download/v2.5.4/proxysql-2.5.4-1-cento
sudo rpm -i proxysql-2.5.4-1-centos8.x86_64.rpm

# Start service
sudo systemctl start proxysql
```

Docker

```
# Pull official image
docker pull proxysql/proxysql:latest

# Run container
docker run -d \
  --name proxysql \
  -p 6032:6032 \
  -p 3306:3306 \
  -v /path/to/proxysql.cnf:/etc/proxysql.cnf \
  proxysql/proxysql:latest
```

Configuration

Configuration File Location

- **macOS**: `/usr/local/etc/proxysql.cnf` or `/opt/homebrew/etc/proxysql.cnf`
- **Linux**: `/etc/proxysql.cnf`
- **Custom**: Pass with `-c` flag: `proxysql -c /path/to/proxysql.cnf`

Basic Configuration Template

```
# ==================================================================
# ProxySQL Configuration File
# ==================================================================

# Admin Variables
admin_variables=
{
  # Admin interface credentials
  admin_credentials="admin:admin"

  # Admin interface listening address and port
  mysql_ifaces="127.0.0.1:6032"

  # Reload configuration from database interval (ms)
  refresh_interval=2000

  # Log file
  datadir="/var/lib/proxysql"
}

# MySQL Variables
mysql_variables=
{
  # Listening interfaces for client connections
  interfaces="0.0.0.0:3306"

  # Maximum number of client connections
  max_connections=2000

  # Default number of connections per user
  default_max_connections=100

  # Minimum number of idle connections to maintain
  connection_max_age_ms=3600000

  # Connection timeout (ms)
  connect_timeout_server_max=30000

  # Query timeout (ms)
  query_timeout=30000
```

```
  # Number of worker threads
  threads=4

  # Enable query caching
  query_cache_size_MB=256
}

# Backend MySQL Servers
mysql_servers=
(
  {
    hostgroup_id=0,
    hostname="db-primary.example.com",
    port=3306,
    weight=1000,
    status="ONLINE",
    max_connections=1000,
    max_replication_lag=10
  },
  {
    hostgroup_id=0,
    hostname="db-replica-1.example.com",
    port=3306,
    weight=1000,
    status="ONLINE",
    max_connections=1000,
    max_replication_lag=10
  },
  {
    hostgroup_id=1,
    hostname="db-replica-2.example.com",
    port=3306,
    weight=1000,
    status="ONLINE",
    max_connections=1000,
    max_replication_lag=10
  }
)

# MySQL Users
mysql_users=
(
  {
```

```
    username="app_user",
    password="secure_password",
    active=1,
    default_hostgroup=0,
    default_schema="myapp_db",
    max_connections=500
  },
  {
    username="analytics_user",
    password="secure_password",
    active=1,
    default_hostgroup=1,
    default_schema="myapp_db",
    max_connections=100
  }
)

# Query Rules
mysql_query_rules=
(
  # Route SELECT queries to read replicas
  {
    rule_id=1,
    match_pattern="^SELECT",
    destination_hostgroup=0,
    apply=1,
    active=1
  },

  # Route INSERT/UPDATE/DELETE to primary
  {
    rule_id=2,
    match_pattern="^(INSERT|UPDATE|DELETE)",
    destination_hostgroup=0,
    apply=1,
    active=1
  },

  # Route specific user queries
  {
    rule_id=3,
    match_pattern="^SELECT",
    username="analytics_user",
```

```
    destination_hostgroup=1,
    apply=1,
    active=1
  },

  # Cache specific queries
  {
    rule_id=4,
    match_pattern="^SELECT.*FROM users",
    cache_ttl=60000,
    apply=1,
    active=1
  }
)
```

Configuration Sections Explained

admin_variables

Controls the admin interface:

```
admin_variables=
{
  admin_credentials="admin:admin"      # Admin login credentials
  mysql_ifaces="127.0.0.1:6032"        # Admin interface port
  refresh_interval=2000                 # Config reload interval (ms)
  datadir="/var/lib/proxysql"          # Data directory location
  logfile="/var/log/proxysql.log"      # Log file path
}
```

mysql_variables

Controls MySQL client connections:

```
mysql_variables=
{
  interfaces="0.0.0.0:3306"            # Listen address and port
  max_connections=2000                  # Max client connections
  default_max_connections=100           # Default per user
  threads=4                             # Worker thread count
  connection_max_age_ms=3600000         # Max connection lifetime
  query_timeout=30000                   # Query timeout (ms)
  wait_timeout=28800000                 # Connection idle timeout
}
```

mysql_servers

Defines backend database servers:

```
mysql_servers=
(
  {
    hostgroup_id=0,                     # Logical group ID
    hostname="db.example.com",          # Server hostname
    port=3306,                          # Server port
    weight=1000,                        # Load balancing weight
    status="ONLINE",                    # Server status
    max_connections=1000,               # Max connections to this server
    max_replication_lag=10              # Max replication lag (seconds)
  }
)
```

mysql_users

Defines application users:

```
mysql_users=
(
  {
    username="app_user",                # Database username
    password="password",                # Password (hashed in runtime DB)
    active=1,                           # Active status (1=yes, 0=no)
    default_hostgroup=0,                # Default hostgroup
    default_schema="database",          # Default database
    max_connections=500,                # Max connections per user
    transaction_persistent=1            # Keep connections within transaction
  }
)
```

mysql_query_rules

Routes queries based on patterns:

```
mysql_query_rules=
(
  {
    rule_id=1,                          # Unique rule ID
    match_pattern="^SELECT",            # Regex pattern to match
    match_digest="SELECT \\*",          # Match hashed query pattern
    username="user",                    # Match specific username
    destination_hostgroup=0,            # Route to this hostgroup
    cache_ttl=60000,                    # Cache timeout (ms)
    log=1,                              # Log query
    apply=1,                            # Apply rule
    active=1                            # Rule is active
  }
)
```

Admin Interface

Connecting to Admin Interface

```
# Default credentials
mysql -h 127.0.0.1 -P 6032 -u admin -p
# Enter password: admin
```

Key Admin Commands

View Configuration

```
-- Show all backends
SELECT * FROM mysql_servers;

-- Show all users
SELECT * FROM mysql_users;

-- Show query rules
SELECT * FROM mysql_query_rules;

-- Show hostgroups
SELECT * FROM mysql_replication_hostgroups;
```

Manage Servers

```sql
-- Add a new server
INSERT INTO mysql_servers (hostgroup_id, hostname, port, weight, status, max_connection
VALUES (0, 'new-db.example.com', 3306, 1000, 'ONLINE', 1000);

-- Modify server
UPDATE mysql_servers
SET weight=500, max_connections=500
WHERE hostname='db.example.com';

-- Remove server
DELETE FROM mysql_servers
WHERE hostname='old-db.example.com';

-- Apply changes
LOAD MYSQL SERVERS TO RUNTIME;
SAVE MYSQL SERVERS TO CONFIG;
```

Manage Users

```sql
-- Add application user
INSERT INTO mysql_users (username, password, active, default_hostgroup, default_schema)
VALUES ('newuser', 'password123', 1, 0, 'myapp_db');

-- Update user
UPDATE mysql_users
SET max_connections=200, default_hostgroup=1
WHERE username='newuser';

-- Remove user
DELETE FROM mysql_users
WHERE username='olduser';

-- Apply changes
LOAD MYSQL USERS TO RUNTIME;
SAVE MYSQL USERS TO CONFIG;
```

Manage Query Rules

```sql
-- Add query rule
INSERT INTO mysql_query_rules
(rule_id, match_pattern, destination_hostgroup, apply, active)
VALUES (10, '^SELECT', 1, 1, 1);

-- Update rule
UPDATE mysql_query_rules
SET cache_ttl=120000
WHERE rule_id=10;

-- Remove rule
DELETE FROM mysql_query_rules
WHERE rule_id=10;

-- Apply changes
LOAD MYSQL QUERY RULES TO RUNTIME;
SAVE MYSQL QUERY RULES TO CONFIG;
```

Query Rules

Rule Matching Options

```
-- Match by query pattern (regex)
match_pattern="^SELECT .*FROM users"

-- Match by query digest (query fingerprint)
match_digest="SELECT * FROM users WHERE id=?"

-- Match by username
username="analytics_user"

-- Match by client address
client_addr="192.168.1.100"

-- Match by schema/database
match_schema="reporting_db"

-- Match by request method
match_method="GET"
```

Routing Examples

Route Read/Write to Different Servers

```
-- Read queries to replica (hostgroup 1)
INSERT INTO mysql_query_rules
(rule_id, match_pattern, destination_hostgroup, apply, active)
VALUES (1, '^SELECT', 1, 1, 1);

-- Write queries to primary (hostgroup 0)
INSERT INTO mysql_query_rules
(rule_id, match_pattern, destination_hostgroup, apply, active)
VALUES (2, '^(INSERT|UPDATE|DELETE)', 0, 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
SAVE MYSQL QUERY RULES TO CONFIG;
```

Route by Username

```sql
-- Analytics user to dedicated replica
INSERT INTO mysql_query_rules
(rule_id, match_pattern, username, destination_hostgroup, apply, active)
VALUES (3, '^SELECT', 'analytics_user', 2, 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
SAVE MYSQL QUERY RULES TO CONFIG;
```

Cache Query Results

```sql
-- Cache user lookups for 5 minutes
INSERT INTO mysql_query_rules
(rule_id, match_pattern, cache_ttl, apply, active)
VALUES (4, '^SELECT .*FROM users WHERE id=', 300000, 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
SAVE MYSQL QUERY RULES TO CONFIG;
```

Block Dangerous Queries

```sql
-- Block DROP and TRUNCATE
INSERT INTO mysql_query_rules
(rule_id, match_pattern, error_msg, apply, active)
VALUES (5, '^(DROP|TRUNCATE)', 'Operation not allowed', 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
SAVE MYSQL QUERY RULES TO CONFIG;
```

Connection Pooling

How Connection Pooling Works

```
Application Connections
         ↓
    ProxySQL (Pooling)
         ↓
Backend MySQL Servers
```

Configuration

```
mysql_variables=
{
  # Maximum total connections
  max_connections=2000

  # Default connections per user
  default_max_connections=100

  # Connection max age (auto-recreate after this time)
  connection_max_age_ms=3600000

  # Idle connection timeout
  wait_timeout=28800000

  # Connection creation timeout
  connect_timeout_server_max=30000
}
```

Monitor Connection Pool

```
-- Connect to admin interface
mysql -h 127.0.0.1 -P 6032 -u admin -p

-- Show connection pool stats
SELECT * FROM stats_mysql_connection_pool;

-- Show detailed pool info
SELECT
  hostgroup_id,
  SUM(ConnUsed) as used,
  SUM(ConnFree) as free,
  SUM(ConnOK) as ok,
  SUM(ConnERR) as errors
FROM stats_mysql_connection_pool
GROUP BY hostgroup_id;

-- Show per-server stats
SELECT
  hostgroup_id,
  hostname,
  port,
  ConnUsed,
  ConnFree,
  ConnOK,
  ConnERR
FROM stats_mysql_connection_pool;
```

Connection Pool Limits

```sql
-- Set per-user limits
UPDATE mysql_users
SET max_connections=500
WHERE username='app_user';

-- Set per-server limits
UPDATE mysql_servers
SET max_connections=1000
WHERE hostname='db-primary.example.com';

-- Apply
LOAD MYSQL USERS TO RUNTIME;
LOAD MYSQL SERVERS TO RUNTIME;
```

Load Balancing

Weight-Based Load Balancing

```
mysql_servers=
(
  {
    hostgroup_id=0,
    hostname="db-primary.example.com",
    port=3306,
    weight=2000,  # 2x more connections
    status="ONLINE"
  },
  {
    hostgroup_id=0,
    hostname="db-secondary.example.com",
    port=3306,
    weight=1000,  # 1x connections
    status="ONLINE"
  }
)
```

Hostgroup-Based Load Balancing

```sql
-- Hostgroup 0: Primary for writes
INSERT INTO mysql_servers
VALUES (0, 'db-primary.example.com', 3306, 1000, 'ONLINE');

-- Hostgroup 1: Replicas for reads
INSERT INTO mysql_servers
VALUES (1, 'db-replica-1.example.com', 3306, 1000, 'ONLINE');
INSERT INTO mysql_servers
VALUES (1, 'db-replica-2.example.com', 3306, 1000, 'ONLINE');

-- Route reads to hostgroup 1
INSERT INTO mysql_query_rules
(rule_id, match_pattern, destination_hostgroup, apply, active)
VALUES (1, '^SELECT', 1, 1, 1);
```

Replication-Aware Load Balancing

```sql
-- Configure replication hostgroups
SELECT * FROM mysql_replication_hostgroups;

-- Set replication monitoring
INSERT INTO mysql_replication_hostgroups
(writer_hostgroup, reader_hostgroup, check_type, comment)
VALUES (0, 1, 'read_only', 'Primary-Replica setup');

LOAD MYSQL SERVERS TO RUNTIME;
```

Monitoring

Connection Statistics

```sql
-- Admin interface
mysql -h 127.0.0.1 -P 6032 -u admin -p

-- Show connection pool stats
SELECT * FROM stats_mysql_connection_pool;

-- Show active sessions
SELECT * FROM stats_mysql_processlist;

-- Show connection totals
SELECT
  hostgroup_id,
  SUM(ConnUsed) as active,
  SUM(ConnFree) as idle
FROM stats_mysql_connection_pool
GROUP BY hostgroup_id;
```

Query Statistics

```sql
-- Top queries by count
SELECT
  digest_text,
  count_star as count,
  sum_time/1000000 as total_time_ms
FROM stats_mysql_query_digest
ORDER BY count_star DESC
LIMIT 10;

-- Slow queries
SELECT
  digest_text,
  sum_time/1000000 as total_ms,
  count_star,
  sum_time/count_star/1000000 as avg_ms
FROM stats_mysql_query_digest
WHERE sum_time/count_star > 1000000  -- > 1 second
ORDER BY sum_time DESC;

-- Query errors
SELECT
  digest_text,
  count_star,
  sum_errors
FROM stats_mysql_query_digest
WHERE sum_errors > 0
ORDER BY sum_errors DESC;
```

Server Health

```sql
-- Show server status
SELECT hostgroup_id, hostname, port, status, ConnUsed, ConnFree
FROM stats_mysql_connection_pool
ORDER BY hostgroup_id, hostname;

-- Show unhealthy servers
SELECT *
FROM mysql_servers
WHERE status != 'ONLINE';

-- Force server status check
MYSQL CHECK SERVERS;
```

Performance Monitoring

```sql
-- Cache hit ratio
SELECT
  (SELECT count_hits FROM stats_cache_requests_total) as hits,
  (SELECT count_misses FROM stats_cache_requests_total) as misses,
  ROUND(100.0 * (SELECT count_hits FROM stats_cache_requests_total) /
    ((SELECT count_hits FROM stats_cache_requests_total) +
     (SELECT count_misses FROM stats_cache_requests_total)), 2) as hit_ratio;

-- Thread stats
SELECT * FROM stats_proxysql_internal;

-- Global stats
SELECT * FROM proxysql_version;
```

Advanced Features

Query Caching

```sql
-- Enable query cache
UPDATE global_variables
SET variable_value='256'
WHERE variable_name='mysql-query_cache_size_MB';

-- Cache specific queries
INSERT INTO mysql_query_rules
(rule_id, match_pattern, cache_ttl, apply, active)
VALUES (10, '^SELECT .*FROM static_data', 600000, 1, 1);

-- Clear cache
PROXYSQL ADMIN VARIABLES LOAD MYSQL VARIABLES FROM CONFIG;

-- Check cache stats
SELECT * FROM stats_cache_requests_total;
```

Query Rewriting

```sql
-- Rewrite queries
INSERT INTO mysql_query_rules
(rule_id, match_pattern, destination_hostgroup,
 query_digest, replace_into, apply, active)
VALUES (20, '^SELECT', 1, 'SELECT.*FROM users',
        'SELECT id, name FROM users /* cached */', 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
```

Traffic Mirroring

```sql
-- Mirror reads to another hostgroup for testing
INSERT INTO mysql_query_rules
(rule_id, match_pattern, destination_hostgroup,
 mirror_hostgroup, apply, active)
VALUES (30, '^SELECT', 0, 2, 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
```

Scheduled Query Caching

```sql
-- Cache results with TTL
INSERT INTO mysql_query_rules
(rule_id, match_pattern, cache_ttl, cache_empty_result,
 cache_ignore_sql_comments, apply, active)
VALUES (40, '^SELECT.*FROM reports', 3600000, 0, 1, 1, 1);

LOAD MYSQL QUERY RULES TO RUNTIME;
```

Troubleshooting

Common Issues

Connection Refused

```bash
# Check if ProxySQL is running
sudo systemctl status proxysql

# Start if not running
sudo systemctl start proxysql

# Check listening ports
sudo netstat -tlnp | grep proxysql

# Check configuration
cat /etc/proxysql.cnf
```

Slow Queries

```sql
-- Find slow queries
SELECT digest_text, sum_time/count_star/1000000 as avg_ms
FROM stats_mysql_query_digest
ORDER BY sum_time DESC
LIMIT 10;

-- Check cache effectiveness
SELECT * FROM stats_cache_requests_total;

-- Increase cache for slow queries
UPDATE mysql_query_rules
SET cache_ttl=600000
WHERE match_pattern='slow_query_pattern';
```

Connection Pool Exhaustion

```sql
-- Check pool status
SELECT * FROM stats_mysql_connection_pool;

-- Increase max connections
UPDATE global_variables
SET variable_value='5000'
WHERE variable_name='mysql-max_connections';

-- Reload
SAVE MYSQL VARIABLES TO CONFIG;
```

Query Routing Issues

```sql
-- Check active rules
SELECT rule_id, match_pattern, destination_hostgroup, active
FROM mysql_query_rules
ORDER BY rule_id;

-- Debug specific query
SELECT
  'Your query' as digest_text,
  destination_hostgroup
FROM mysql_query_rules
WHERE 'Your query' REGEXP match_pattern
LIMIT 1;
```

Log Files

```bash
# View ProxySQL logs
tail -f /var/log/proxysql.log

# Filter for errors
grep ERROR /var/log/proxysql.log

# Monitor in real-time
sudo journalctl -u proxysql -f
```

Reset Configuration

```bash
# Backup current config
cp /etc/proxysql.cnf /etc/proxysql.cnf.backup

# Reset to defaults
rm /var/lib/proxysql/proxysql.db
sudo systemctl restart proxysql
```

Performance Tips

1. **Tune Thread Count**: Set `threads` based on CPU cores
2. **Connection Pooling**: Use aggressive pooling for high-traffic apps
3. **Query Caching**: Cache read-heavy queries with long TTL
4. **Hostgroup Routing**: Separate read/write traffic
5. **Weight Distribution**: Balance load by server capacity
6. **Monitor Actively**: Track connection pool and query stats
7. **Rule Order**: Order rules by specificity (specific before general)
8. **Connection Reuse**: Use persistent connections from app

Example: Complete Setup

```
# /etc/proxysql.cnf
admin_variables=
{
  admin_credentials="admin:admin"
  mysql_ifaces="127.0.0.1:6032"
  refresh_interval=2000
}

mysql_variables=
{
  interfaces="0.0.0.0:3306"
  max_connections=2000
  default_max_connections=100
  threads=4
  query_timeout=30000
}

mysql_servers=
(
  { hostgroup_id=0, hostname="db-primary.example.com", port=3306, weight=1000, status="
  { hostgroup_id=1, hostname="db-replica-1.example.com", port=3306, weight=1000, status
  { hostgroup_id=1, hostname="db-replica-2.example.com", port=3306, weight=1000, status
)

mysql_users=
(
  { username="app_user", password="secure_password", active=1, default_hostgroup=0, def
)

mysql_query_rules=
(
  { rule_id=1, match_pattern="^SELECT", destination_hostgroup=1, apply=1, active=1 },
  { rule_id=2, match_pattern="^(INSERT|UPDATE|DELETE)", destination_hostgroup=0, apply=
  { rule_id=3, match_pattern="^SELECT.*FROM users WHERE id=", cache_ttl=300000, apply=1
)
```